# TIES-Merging Notebooks: Comparative Analysis

ties-merging-fast-1 → ties-merging-fast-4

## 1 Overview

The four files represent successive iterations of the *same* pipeline, each one responding to a problem diagnosed in the previous run. The UNet/DDPM architecture code (cell 1 in notebooks 1–3, and the `architecture.py` module in notebook 4) is byte-for-byte identical across all four notebooks — only the configuration, merge logic, evaluation protocol, and search strategy change.

## 2 Notebook-by-notebook summary

### 1. `ties-merging-fast-1.ipynb` — baseline pipeline

The first, "clean" version of the pipeline. Seven cells: architecture, configuration, merge functions (`trim_global`, sign election, TIES merge with `mean` aggregation only), evaluation functions, a two-stage grid search (proxy $v$-loss ranking → PSNR/SSIM + retention on the top-$K$), final evaluation, and visualization. The grid search only sweeps `keep_ratio` $\in \{0.1, 0.2, 0.3, 0.5\}$, `lambda_scale` $\in \{0.4, 0.6, 0.8, 1.0, 1.3\}$ and three `task_lambdas` settings, all combined with the classical TIES `mean` rule. Grid search and evaluation both run on the validation splits of the three training manifests.

### 2. `ties-merging-fast-2.ipynb` — diagnostic pass

Structurally identical to notebook 1 (same 7 cells, same numbers wherever they overlap), but with one extra cell inserted after the evaluation functions: a **diagnostic block** that stress-tests four hypotheses for why the merge underperforms:

- **H1** – the merge itself is the problem (task-vector interference);
- **H2** – the specialists are undertrained;
- **H3** – there is a code/sampler bug;
- **H4** – the evaluation metric is uninformative.

It runs a trivial-baseline test (identity mapping vs. target), and a DDIM-vs-native-sampler comparison for each specialist. The conclusion, visible in the printed output, is that **all three specialists work correctly** (e.g. star removal reaches 40.6 dB with either sampler, far above the 23.4 dB identity baseline) — which rules out H2/H3/H4 and isolates the problem to the merge step itself (H1). This diagnosis is what motivates notebook 3.

### 3. `ties-merging-fast-3.ipynb` — "v5", addressing H1

Rewritten based on the notebook-2 diagnosis. Key changes:

- A new `merge_method` knob: besides classical `mean` (which divides the merged contribution by the number of agreeing tasks and was identified as diluting the signal), it adds `sum` (preserves task-vector magnitude) and `dropout_rescale` (DARE-style trim + rescale by 1/keep_ratio).
- A stronger grid: `keep_ratio` up to 0.8, `lambda_scale` up to 1.6, because a weak $\theta_0$ base (only ∼12–23 dB before merging) needs larger corrections.

- Selection criterion changed to mean retention with *min*-retention as a tie-break, to avoid rewarding configurations that sacrifice one task entirely.
- All `dataset_merged`-related code removed; evaluation is exclusively on the three manifest validation splits (same protocol as 1/2, narrower scope).

In practice only `merge_method="sum"` was actually swept (the other two options are present in code but commented out of the grid), over `keep_ratio`$\in \{0.2, 0.5, 0.8\}$ and `lambda_scale`$\in \{0.5, 0.8\}$ (15 configurations total). The notebook completed end-to-end and saved `ties.pth`.

### 4. `ties-merging-fast-4.ipynb` — "v6", task-arithmetic rewrite, multi-GPU

A structural rewrite rather than an incremental patch. Instead of one monolithic notebook it *generates a Python package* (`/kaggle/working/ties_merge/{architecture,configuration,merging,evaluation,search}.py`) via `%%writefile` cells, and launches the search as a separate multi-GPU process with `accelerate launch -num_processes=2`. Design changes:

- **Dataset**: abandons the training manifests in favor of the `dataset_merged/{SU,IR,SR,...}` folders (same data `task-arithmetic` uses), with a `subsample_pct` (10 %) for speed, read through a new `DegradedImageDataset`.
- **Two-stage search with a different metric**: Stage 1 evaluates every `(merge_method, task_lambdas, keep_ratio, lambda_scale)` combination on the three *single-task* folders using a normalized score score = $\text{mean}_t(\text{MSE}_{\text{merge},t}/\text{MSE}_{\text{expert},t})$, discarding any config whose per-task ratio exceeds `stage1_max_task_ratio = 2.5` (a "safe zone" filter). Stage 2 was intended to re-rank the surviving "safe" configs on *multi-degradation* folders (`IR_SU`, `SR_SU`, `SR_IR`, `SR_IR_SU`) by mean MSE, and save the winner.
- TIES machinery itself (task vector, global trim, sign election, `merge_method`) is unchanged from notebook 3; only the search protocol and dataset source come from `task-arithmetic`.

**Execution outcome:** Stage 1 completed successfully (15/15 configurations evaluated, 2 passed the safe-zone filter). Stage 2 crashed immediately with

## 3 Cross-cutting differences

- **Architecture**: identical in all four (UNet + DDPM, v-prediction, 200 diffusion steps, EMA weights preferred).
- **Merge core (trim + sign election)**: unchanged since notebook 1; what changes is how the per-task, sign-agreed deltas are *combined* (`mean` only in 1/2, `mean`/`sum`/`dropout_rescale` available from 3 onward, though only `sum` is swept in 3 and 4).
- **Evaluation data**: notebooks 1–3 use the *training manifests'* validation splits (same distribution as training, in-distribution generalization check). Notebook 4 switches to the held-out `dataset_merged` folders, including genuinely novel multi-degradation combinations — closer to the original stated goal of testing composition of unseen degradations, but this is also where it fails.
- **Search protocol**: 1/2/3 use a 2-stage proxy-$v$-loss → PSNR/SSIM-retention search over the *same* single-task validation data. 4 uses a differently-motivated 2-stage search (single-task safe-zone filter → multi-task re-ranking) over a *different* dataset per stage.
- **Selection metric**: retention relative to the specialist (1, 2, 3) vs. MSE ratio relative to the expert, unnormalized by $\theta_0$ (4).
- **Infrastructure**: 1–3 are single-process, in-notebook. 4 is a generated multi-file package executed via `accelerate` on 2 GPUs, which is also the source of its failure (Stage-2 dataset path only checked at run time, in a separate process, uncaught by any prior validation cell).
- **Deliverable**: 1, 2, and 3 each produce a working `ties.pth` and a completed visualization cell. 4 does not produce a merged checkpoint in the run captured in this notebook, because Stage 2 never completes.

# 4 Summary table

| Notebook | Key design change vs. previous | Search / metric | Captured output |
|---|---|---|---|
| **1** (baseline) | Reference TIES pipeline. Only `mean` merge rule. | 2-stage: proxy $v$-loss $\rightarrow$ PSNR/SSIM + retention, on manifest val splits. | Ran end-to-end. $\theta_0$: 23–25 dB; specialists 26–41 dB. Merged model: restoration 28.99 dB, super-res 23.50 dB, star removal 21.82 dB (SSIM 0.55–0.88) — **below $\theta_0$ on every task** (deltas $-0.97$ to $-3.63$ dB). |
| **2** (diagnosis) | Same pipeline as 1 (identical numbers), plus an inserted **diagnostic cell** (H1–H4 hypothesis tests). | Same as 1, plus trivial-baseline & DDIM-vs-native sampler tests. | Identity baselines 23–25 dB; specialists 26–41 dB with *either* sampler $\Rightarrow$ **specialists confirmed healthy**; problem isolated to the merge (H1). Final merge numbers identical to notebook 1. |
| **3** ("v5") | New `merge_method` (`sum`/`dropout_rescale` added to `mean`); wider grid (`keep_ratio`$\leq$0.8, $\lambda \leq$1.6); `dataset_merged` code removed. | Same 2-stage protocol as 1/2, but only `sum` swept; selection = mean retention, min-retention tie-break. | Ran end-to-end (15 configs, $\sim$315 s). Best: `sum`, keep=0.8, $\lambda$=0.5 $\rightarrow$ PSNR restoration 31.6 dB, super-res 23.5 dB, star removal 24.9 dB; retention mean $-0.18$, min $-1.09$ (still negative overall, but a clear improvement over notebook 1's merge, especially for restoration). |
| **4** ("v6", best design) | Full rewrite as a Python package + `accelerate`, multi-GPU. Dataset switched to `dataset_merged` folders (incl. true unseen multi-degradation combos, e.g. `SR_IR_SU`); normalized MSE-ratio scoring; safe-zone filter (ratio $\leq$2.5) before Stage 2. | Stage 1: MSE ratio vs. expert on single tasks, 15 configs. Stage 2 (intended): mean MSE on 4 multi-task folders. | Stage 1 completed: best safe configs at keep=0.5/0.8, $\lambda$=0.5 (score 1.00 / 1.20); expert baselines 21–37 dB. **Stage 2 crashed** (`FileNotFoundError` on folder `IR_SR` vs. configured `SR_IR`) $\Rightarrow$ no `ties.pth` saved; visualization cell fails. Design is the most complete/correct in principle, but **this run did not finish**. |

Table 1: Comparison of the four TIES-merging notebook iterations: design changes, search/evaluation protocol, and the numbers actually produced by the captured run of each notebook.

# 5 Note on "notebook 4 is the best"

By **design**, notebook 4 is the most advanced iteration: it evaluates on genuinely held-out data (including multi-degradation combinations never seen by any single specialist), uses a cleaner normalized scoring function, adds a safe-zone filter against destructive merges, and is engineered to scale to multi-GPU search. That said, the run captured in this file did *not* complete: Stage 2 fails on a folder-naming mismatch (`tasks_stage2` configured as `SR_IR` vs. an on-disk/expected folder `IR_SR`) before any multi-task score is computed, so no merged checkpoint (`ties.pth`) exists at the end of the notebook and the final visualization cell errors out. Fixing that one path (or renaming the folder to match) is a small change, and would let Stage 2 – and therefore the qualitatively strongest version of the pipeline – actually run to completion. Notebook 3 is, among the four, the

only later iteration with both an improved merge strategy *and* a completed, saved output, making it the best *currently working* checkpoint, while notebook 4 is the best *design* once its Stage-2 path bug is fixed.